CO2 Assessment 02 Date: 24-10-2026

SCENARIO BASED

Course Code: DSA0308

Course Name: Natural Language Processing

Register No: 192324299 Haritha S

Question 1. Morphology-Based Normalization

Aim

To develop a Python program that decomposes the words “govern”, “government”, and “governance” into root words and derivational suffixes, identifies their derivational levels, and maps them to a common representation for topic modeling and document clustering.

Morphological breakdown:

Original word	Root	Affix	Derivational level	Normalized form
govern	govern	None	Level 0	govern
government	govern	-ment	Level 1	govern
governance	govern	-ance	Level 1	govern

Code:

```
words = ["govern", "government", "governance"]

suffixes = ["ment", "ance"]

def analyze_word(word):

    root = word

    detected_affixes = []

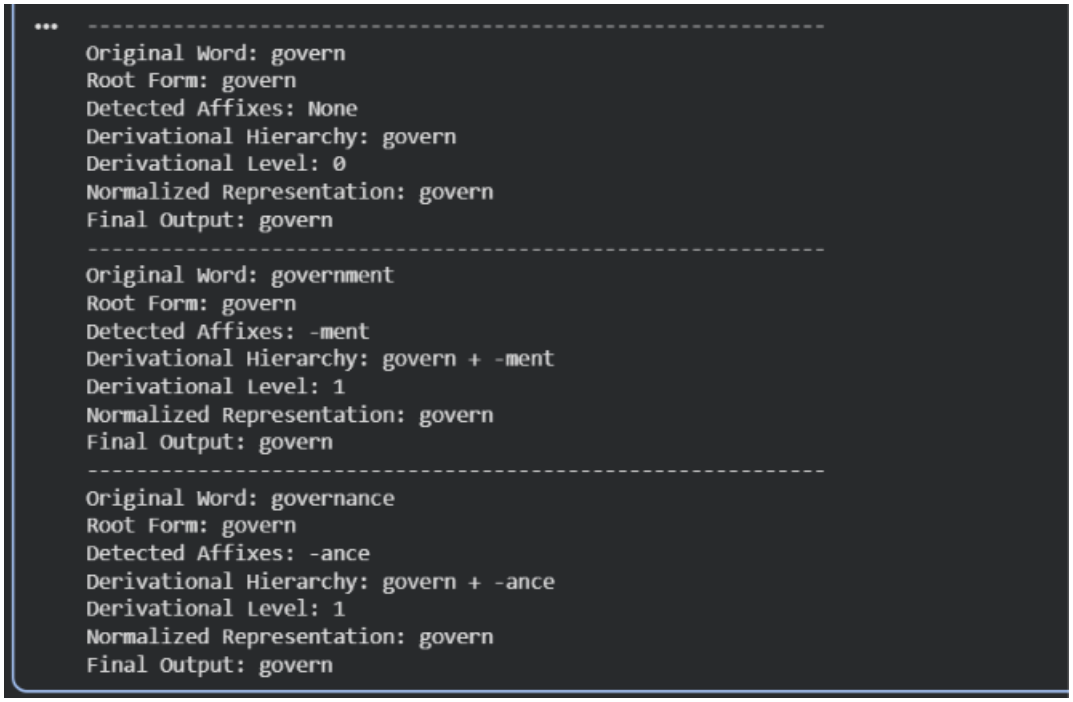
    hierarchy = []

    for suffix in suffixes:
```

```
        if root.endswith(suffix) and len(root) > len(suffix):
            root = root[:-len(suffix)]
            detected_affixes.append("-" + suffix)
            hierarchy.append(root + " + " + "-" + suffix)
            break
    level = len(detected_affixes)
    normalized = root
    return {
        "Original Word": word,
        "Root Form": root,
        "Detected Affixes": (
            ", ".join(detected_affixes)
            if detected_affixes else "None"
        ),
        "Derivational Hierarchy": (
            " -> ".join(hierarchy)
            if hierarchy else root
        ),
        "Derivational Level": level,
        "Normalized Representation": normalized,
        "Final Output": normalized
    }
results = [analyze_word(word) for word in words]
for result in results:
    print("-" * 60)
    for key, value in result.items():
```

```
    print(f"{key}: {value}")
```

Output:



Result:

The morphology-based normalization program successfully identifies the root govern and the suffixes -ment and -ance. It assigns derivational level 0 to govern and level 1 to government and governance. All three words are mapped to the common normalized representation govern, making them easier to group during topic modeling and document clustering.

Question 2. Morphological Parsing and Normalization

Aim

To develop a Python program that identifies prefixes, roots, and suffixes in the words activate, activation, and reactivation, analyzes their derivational transformations, and maps them to a common base form for document classification and semantic indexing.

Morphological analysis

Original word	Prefix	Root/base	Suffix	Word class	Normalized form
activate	None	activate	None	Verb	activate
activation	None	activate	-ion*	Noun	activate
reactivation	re-	activate	-ion*	Noun	activate

Code:

```
words = ["activate", "activation", "reactivation"]

def analyze_word(word):
    prefix = "None"
    suffix = "None"
    root = word
    derivation = []
    meaning = ""
    word_class = ""

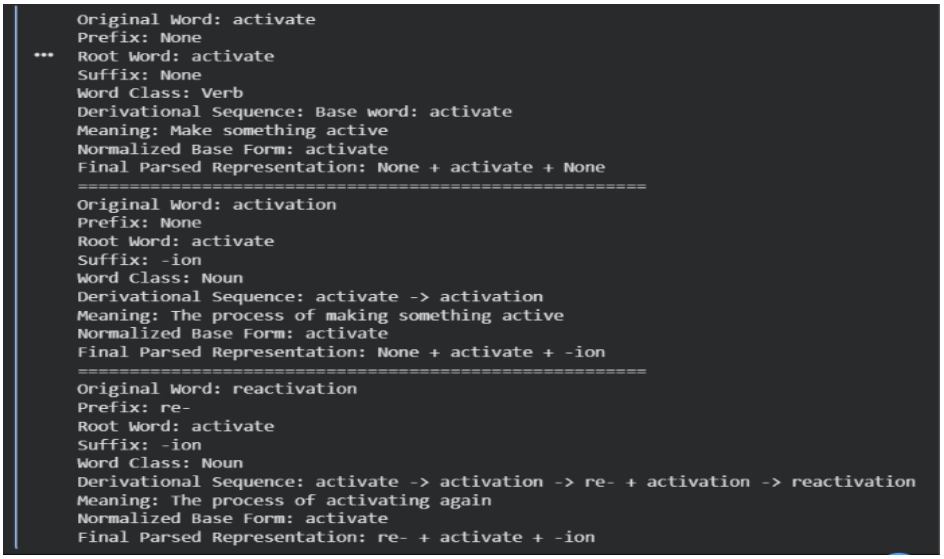
    if root.startswith("re"):
        prefix = "re-"
        root = root[2:]
        derivation.append("re- + activate")
        meaning = "Activation again"
```

```
else:
    meaning = "Make something active"
if root == "activate":
    word_class = "Verb"
    normalized = "activate"
elif root == "activation":
    suffix = "-ion"
    root = "activate"
    word_class = "Noun"
    derivation.append("activate -> activation")
    normalized = "activate"
elif root == "activation":
    suffix = "-ion"
    root = "activate"
    word_class = "Noun"
    derivation.append("activate -> activation")
    normalized = "activate"
if word == "reactivation":
    prefix = "re-"
    root = "activate"
    suffix = "-ion"
    word_class = "Noun"
    derivation = [
        "activate -> activation",
        "re- + activation -> reactivation"
    ]
```

```
        normalized = "activate"
        meaning = "The process of activating again"
    elif word == "activation":
        root = "activate"
        suffix = "-ion"
        derivation = ["activate -> activation"]
        meaning = "The process of making something active"
    elif word == "activate":
        root = "activate"
        derivation = ["Base word: activate"]
        meaning = "Make something active"
    return {
        "Original Word": word,
        "Prefix": prefix,
        "Root Word": root,
        "Suffix": suffix,
        "Word Class": word_class,
        "Derivational Sequence": " -> ".join(derivation),
        "Meaning": meaning,
        "Normalized Base Form": normalized,
        "Final Parsed Representation":
            f"{prefix} + {root} + {suffix}"
    }
for word in words:
    result = analyze_word(word)
    print("=" * 55)
```

```
    for key, value in result.items():
        print(f"{key}: {value}")
```

Output:



Result:

The program demonstrates how prefixes and suffixes change a word's meaning and grammatical category. All three input words are mapped to the normalized base form activate, helping a document classification system group related terms under a common semantic concept.